

WavePrep - Create Machine-Learning-Ready Datasets from MIMIC Waveforms

M. Elwes, O. Beyan, E. Kutafina

Abstract—The Medical Information Mart for Intensive Care (MIMIC) waveform databases provide high-resolution physiological signals from intensive care units but remain underutilised compared to the relational MIMIC core databases. One barrier is the lack of accessible tools for transforming waveform data stored in the Waveform Database (WFDB) format into machine-learning-ready datasets. This work introduces WavePrep, an open-source software framework that addresses this gap by enabling flexible, transparent, and efficient preprocessing of MIMIC waveform data. WavePrep allows users to define fine-grained, channel-specific preprocessing pipelines via a user-friendly JSON configuration, supporting common signal processing steps such as downsampling, data cleaning, imputation, windowing, and dataset splitting. The framework automatically detects dependencies between preprocessing steps, prevents data leakage through patient-level splitting, and exploits parallelisation to accelerate dataset creation. We evaluate WavePrep by creating multiple datasets derived from the MIMIC-III matched waveform database, including numeric and non-numeric signals. Experimental results on a multi-core server demonstrate robust and scalable performance, with processing times between 33.10 and 50.53 milliseconds per hour of recording. Datasets created using WavePrep were successfully applied in machine learning tasks. By lowering technical barriers and providing transparent preprocessing metadata, WavePrep facilitates broader adoption of clinical waveform data and improves the AI-readiness of signal datasets. GitHub: <https://github.com/BI-K/WavePrep>

Index Terms—intensive care, MIMIC, AI-ready, open source software, signal processing, biosignal

I. INTRODUCTION

The Medical Information Mart for Intensive Care (MIMIC) is a well-established and valuable resource for researchers working with data from intensive care units (ICUs). There are multiple versions of MIMIC, each containing data recorded over different time frames; the most recent is MIMIC-IV, published in 2023. The MIMIC databases II-IV consist of two parts: "MIMIC core" and "MIMIC waveform". The MIMIC core consists of a relational database containing various chart events, including vital signs and medications, as well as patient information. MIMIC waveform contains high-resolution bedside patient monitoring. Different levels of resolution are available: *numeric* waveforms contain recordings such as heart rate, oxygenation level, or blood pressure at 1-per-minute measurements; *non-numeric* waveforms contain higher-resolution recordings, such as plethysmography. The "MIMIC matched waveform" database allows retrieval of additional information about a patient's stay by matching *patient_id* to the recording

date. These waveforms are stored in the Waveform Database (WFDB) format, which is widely used in the PhysioNet [1] datasets.

Despite widespread adoption of core MIMIC databases, MIMIC waveform databases are comparably underutilised. Although the core MIMIC-III [2] has 98.000 citations on Google Scholar as of December 2025, compared to 124 citations of the MIMIC-III waveform [3]. In contrast to core MIMIC, MIMIC waveforms are publicly available without the need to apply for credential access. One reason for the discrepancy between the use of the core MIMIC databases and the MIMIC (matched) waveform databases could be researchers' unfamiliarity with the WFDB format rather than with relational databases.

To use a dataset for machine learning, it first needs to be made machine learning-ready, or, more broadly *AI-ready*. This includes adequate preprocessing steps and proper splitting into training and test sets. Apart from adequate preprocessing, an AI-ready dataset should be accompanied by metadata that allows researchers to assess its suitability for a research question. These metadata should include basic information (data ownership, version, data type), data quality information, and data provenance, including a report on all applied processing steps [4]. Standard pre-processing steps for waveform data include down- or up-sampling, data cleaning, imputing missing values, and windowing. Depending on the use case, these steps can be combined in various ways and may differ across channels of the same recording. To our best knowledge, there is no published tool for creating a machine-learning-ready dataset from WFDB databases.

In this paper, we propose WavePrep, an open-source software that provides a flexible pipeline for creating AI-ready datasets from the *numeric* and *non-numeric* MIMIC-III waveform database. In particular, we provide a user-friendly JSON interface that allows users to define preprocessing steps with fine-grained, step-by-step, and channel-specific control, perform a train-test split, and enable automatic parallelisation to speed up dataset creation significantly.

II. METHODS

To use WavePrep, the user must provide two input files; an impression is given in Figure 1. Firstly, a CSV file specifying the selected records, along with the time span for each record that should be included in the resulting dataset. Secondly, a JSON file specifying the channels to extract from the record, the channel-specific pre-processing steps, the windowing, and the split into train, test, and validation sets.

WavePrep iterates over the channel-specific steps, as shown in Algorithm 1, and supports three categories of processing

M. Elwes, O. Beyan and E. Kutafina are with the University of Cologne, Faculty of Medicine and University Hospital Cologne, Institute for Biomedical Informatics, Germany, e-mail: mayra.elwes@uk-koeln.de.

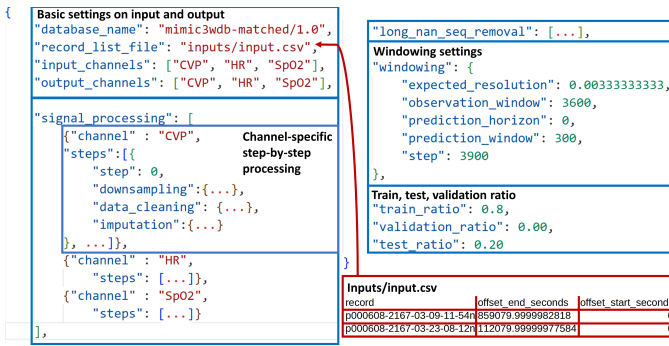


Fig. 1: Shortened exemplary config.json and input.csv, detailed descriptions are provided in the GitHub repository.

steps: downsampling, data cleaning, and imputation. In particular, supported downsampling strategies include decimation, mean, and median downsampling. Data cleaning is supported in two ways. Firstly, filtering by replacing specific values with NaN. Currently, filtering is supported via channel-specific user-defined valid value ranges. Secondly, by removing sequences of the record in which at least one channel has a sequence of NaN values longer than a user-specified cut-off sequence length. A combination of filtering and removal of long NaN sequences can be used to remove sequences of implausible data, e.g., when a sensor was not connected to the patient. For imputation, only one strategy is currently supported: forward filling the first three missing values and then using an iterative imputer to fill the remaining values.

Algorithm 1: WavePrep - Signal Processing

```

Data: config, start_step, end_step,  $S$ 
Result:  $S$ 
for  $step\_idx \leftarrow start\_step$  to  $end\_step$  do
  foreach  $channel \in specified\_channels$  do
    for  $i \in len(S)$  do
       $channel\_data \leftarrow S[i][channel]$  if  $step\_idx \in$ 
         $config("channel\_specific\_steps")$  then
         $channel\_data \leftarrow chan-$ 
           $nel\_specific\_steps[step\_idx](channel\_data,$ 
             $config);$ 
        end
      end
    end
  end
  if  $step\_idx \in config("remove\_nan\_sequences")$ 
    then
     $S \leftarrow remove\_nan\_sequences(S, config)$ 
  end
end

```

WavePrep automatically determines when a preprocessing step, such as iterative imputation, requires defined train-test division, and will window and split the dataset before executing it. WavePreps algorithm is shown in Algorithm 2. Windowing is done based on four parameters: the duration of the observation window, the prediction horizon, the prediction window, and an offset to the next window. The dataset split is performed based on user-specified train-, validate- and test-ratios. To avoid data leakage, a group-shuffle split at the patient

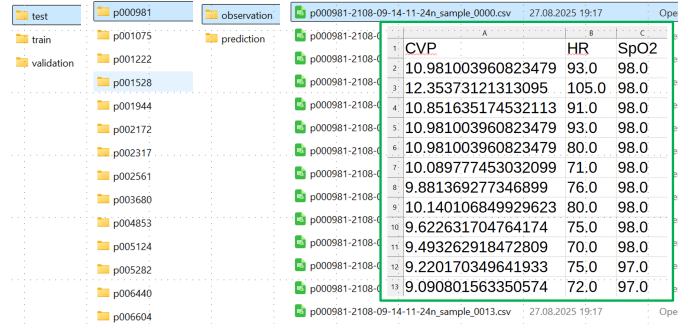


Fig. 2: Structure of WavePrep output.

level is applied.

Algorithm 2: WavePrep - Overarching Process

```

Data: records_start_end, config
Result: processed samples in csv format
start_step, end_step, max_step  $\leftarrow$  get_steps(config)
parallel for record, start_offset, end_offset  $\in$ 
records_start_end do
     $S \leftarrow$  fetch_wfdb_record_snippet;
     $S \leftarrow$  signal_processing_(config, start_step,
        end_step,  $S$ );
    windows  $\leftarrow$  windowing( $S$ , config);
    store_windows_to_disc(windows, config);
end
split(config);
if imputer needs to be trained then
    train imputer and store to disc;
end
if end_step  $\leq$  max_step then
    for split_type  $\in$  { "train", "validate", "test" } do
        parallel for folder in folders in
        folders(split_type) do
             $S \leftarrow$  fetch and transform data from disc;
             $S \leftarrow$  signal_processing_(config, end_step,
                overall_max_steps,  $S$ );
            store_to_disc( $S$ , config);
        end
    end
end

```

The output of WavePrep is CSV files of the observation and prediction windows, is organised in a specific file structure, as shown in Figure 2. At the first level, the resulting dataset is split into *train*, *validation*, and *test* sets. Within these folders, the processed samples are grouped by patient and named according to the rule *record_id_sample_number.csv*. All observation windows of a patient’s recordings are grouped under that patient’s *observation* folder; likewise, the prediction windows are grouped under the *prediction* folder. Within the CSV files, one column corresponds to one channel, and each row corresponds to a timepoint.

WavePrep is developed in Python 3.12.3. It mainly depends on WFDB, Pandas, NumPy, and scikit-learn. The preprocessing step categories are implemented as abstract classes, enabling researchers to extend the framework with specific implementations of these steps. Records are processed in

TABLE I: Performance of WavePrep.

Data-set	# Records	Total duration of records [s]	Processing time [s]	Avg time / record [s]	Avg time / hour of record [ms]
1	1092	78657.67	2603.75	2.38	33.10
2	945	54629.98	1853.87	1.96	33.94
3	456	42522.57	1595.46	3.50	37.52
4	512	36965.40	1430.72	2.79	38.70
5	565	35293.08	1414.06	2.50	40.07
6	445	21335.88	889.84	2.00	41.71
7	440	23041.47	968.62	2.20	42.04
8	241	14184.00	716.69	2.97	50.53

parallel.

III. RESULTS

We assess WavePrep’s processing speed by creating eight datasets. All records are selected from the *numeric* records of MIMIC-III matched waveform v1.0. The selected records and time spans specified in input.csv differ across datasets, and the number of recordings and total duration to be processed for each dataset are shown in Table I. The preprocessing steps specified in config.json are the same for all datasets.

The preprocessing steps specify extracting data for six channels. Each of these channels is processed in the same way: (1) mean downsampling to 1/1 minute, (2) exclusion of data outside of a specified range, (3) imputation via an iterative imputer, and (4) finally mean downsampling to 1/5 minutes. In this configuration, WavePrep automatically detects that, to apply step (3), an imputer needs to be trained on the test dataset’s observation windows. So it performs windowing and train-test split before step (3). For windowing, the recordings are sliced into 60-minute observation windows, 0-minute prediction horizons, and 5-minute prediction windows, in a non-overlapping manner.

The experiments are performed on a Server with two Intel Xeon Silver 4316 (40 cores, 80 threads @2.30GHz). The results of the experiments are shown in Table I. The average processing time per record is between 1.96 and 3.50 seconds, and per hour of recording, between 33.10 and 50.53 milliseconds. All encountered records were processed correctly.

Compatibility with the *non-numeric* records of MIMIC III matched waveforms was confirmed via preprocessing that included downsampling and data cleaning of the plethysmography channel over the first 60 seconds of eight records.

WavePrep was successfully used to prepare various datasets for a study benchmarking AI models for vital parameter prediction tasks [6], demonstrating its ability to create machine-learning-ready datasets.

IV. DISCUSSION

WavePrep generated the datasets for the MIMIC-III matched waveform v1.0 experiments as expected, demonstrating its ability to preprocess data to meet the needs of a machine learning project. By transforming data from WFDB to CSV, a format already familiar to many researchers, WavePrep

may lower the barrier to working with the MIMIC waveform databases. The processing time per recorded hour decreases with increasing number of records to process, due to parallelisation in WavePrep. The open-source nature of the tool and the use of abstraction enable the community to adapt and further develop WavePrep to their specific pre-processing needs. The processing step categories are limited to downsampling, data cleaning, and imputing, for now, but can be expanded following the abstraction pattern to include further pre-processing categories. Using WavePrep addresses AI-readiness requirement through metadata by providing the config.json and input.csv files, which detail the processing and data split applied to the dataset.

V. CONCLUSION

WavePrep is a tool for creating machine-learning-ready signal datasets from MIMIC waveform databases. The tool allows for configuring pre-processing in a flexible, fine-grained, and channel-specific manner, supporting common signal pre-processing steps, such as downsampling, data cleaning, cutting out long NaN-sequences, imputation, windowing, and splitting a dataset into training and test data. These steps can be specified via a user-friendly JSON file, lowering the entry barrier to using MIMIC waveform data. An exemplary performance analysis on a server with 80 available workers showed that the tool takes between 33.10 and 50.53 milliseconds to process 1 hour of recording when processing 6 channels. The modular, open-source tool enables researchers to add functionality according to their needs. In future work, we will extend WavePrep with a detailed dataset quality report, further addressing AI-readiness and helping researchers create high-quality signal datasets with minimal coding. Additionally, we will integrate the Croissant metadata format for datasets [5], addressing AI-readiness demands of the machine learning community.

REFERENCES

- [1] A. Goldberger, A. Amaral et.al. *PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals*. Circulation [Online], 2000.
- [2] A. E. W. Johnson et al., *MIMIC-III, a freely accessible critical care database*. Sci Data, vol. 3, no. 1, p. 160035, May 2016. doi:10.1038/sdata.2016.35
- [3] B. Moody, G. Moody, M. Villarreal, G. Clifford, and I. Silva, *MIMIC-III Waveform Database Matched Subset (version 1.0)*, PhysioNet. RRID:SCR_007345, 2020. doi: 10.13026/c2294b
- [4] S. Afzal, C. Rajmohan, M. Kesarwani, S. Mehta, and H. Patel, *Data Readiness Report*. In: 2021 IEEE International Conference on Smart Data Services (SMDS), Chicago, IL, USA: IEEE, Sep. 2021, pp. 42–51. doi: 10.1109/SMDS53860.2021.00016
- [5] M. Akhtar et al., *Croissant: A Metadata Format for ML-Ready Datasets*. In: Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning. Santiago AA Chile: ACM; 2024. p. 1–6. doi: 10.1145/3650203.3663326
- [6] M. Elwes et al., *Distribution Shift Analysis in Generalizable Modelling: Intensive Care Time-Series Data*, Submitted for publication at MIE 2026. doi: 10.13140/RG.2.2.14589.83684